

CurL-AutoML: Curriculum Learning-based AutoML

Lucas Nildaimon dos Santos Silva^{*†}, Lucas Cardoso Silva^{*†}, Fernando Rezende Zagatti^{*†},
Bruno Silva Sette^{*†}, Helena de Medeiros Caseli[†], Daniel Lucrédio[†] and Diego Furtado Silva[†]

^{*}Americanas S.A. - Digital Lab, São Paulo, SP, Brazil

[†]Federal University of São Carlos - Department of Computing, SP, Brazil

Email: lucas.silva@estudante.ufscar.br, lucascardoso@estudante.ufscar.br, fernando.zagatti@estudante.ufscar.br, bruno.sette@ufscar.br, helenacaseli@ufscar.br, daniel.lucradio@ufscar.br, diegofs@ufscar.br

Abstract—AutoML aims to find the best Machine Learning (ML) pipeline in a complex and high-dimensional search space by evaluating multiple algorithm configurations. However, training multiple ML algorithms is time-consuming, and as AutoML tools are frequently time-constrained, the exploration of the search space may find sub-optimal results. In this work, we explore the application of curriculum learning techniques to overcome this limitation. Curriculum and anti-curriculum learning have improved model performance and accelerated the training process on previous empirical investigations using optimization-based models by ordering examples during model training based on their difficulty. We apply and compare curriculum strategies on an AutoML system to accelerate the search space exploration and find good-performing machine learning pipelines efficiently. The results indicate that AutoML can benefit from a curriculum strategy. Furthermore, in most of the evaluated scenarios, the curriculum strategies led to better classification results.

Index Terms—Automated Machine Learning, Curriculum Learning, Hyper-Parameter Optimization.

I. INTRODUCTION

Defining a ML pipeline takes substantial human expertise and usually requires several model training hours to find the right combination of components and hyperparameters for each dataset [1].

Given the lack of a single algorithm capable of achieving excellent performance in any application, researchers continuously propose a plethora of new methods or modifications of existing algorithms for specific tasks. With such an increase in algorithms and problem variability, the ML process becomes even more complex, expensive, and dependent on high human expertise and skills.

Motivated by industry needs and machine learning (ML) challenges in general, Automated Machine Learning [2] (AutoML) has emerged in recent years as a sub-area of ML. The goal of AutoML is to reduce the need for human intervention in the ML process, making it more accessible for non-expert users and less expensive for the industry.

Current AutoML approaches are competitive with humans in some scenarios. However, most AutoML approaches rely on robust hyperparameter optimization and algorithm selection techniques to search through a space that includes different algorithms and configurations. It results in a complex, conditional, and high dimensional search space [2], making it costly for AutoML to solve.

According to Parmentier et al. [3], one of the reasons why AutoML is so time-consuming is due to the time taken by training the ML algorithm. Training ML algorithms on small datasets takes at least a few seconds and up to days for bigger datasets, which is increasingly common with big data.

Inspired by the way humans learn, Curriculum Learning (CL) adapts the concept of curriculum to ML algorithms. In human education, teachers rely heavily on a highly organized curriculum, starting from easy concepts and gradually presenting more complex ones. Following a similar idea, CL proposes to train ML algorithms by presenting first a small subset of easier data and gradually increasing both the subset size and the difficulty of data within it [4], [5]. Previous studies show that CL can improve model performance on target tasks and accelerate the training process [6]. Despite being counter-intuitive, anti-curriculum learning (AntiCL), that is, training a ML model from the hardest to the easiest data, proved to be as good or better than CL in some scenarios [7], [8].

Hence, in this work, we propose and compare different CL strategies on an AutoML system to accelerate the search space exploration and find good performing machine-learned models with efficiency. Unlike previous work by [9], which utilizes CL strategies to speed up neural architecture search (NAS), our focus is on AutoML for “traditional” ML. That is, ML pipelines that do not include deep neural networks.

The typical approach for applying CL consists of two main components: (i) a difficulty measurer, for estimating the difficulty of each example in the training set, and (ii) a training scheduler that decides the pace and the sequence of data subsets to be presented throughout the training process [6]. In terms of difficulty measurers, we used two methods based on instance hardness [10], which estimate instances that are hard to classify correctly. We adapted the Successive Halving optimization technique present in the Auto-Sklearn system [11] to sample data subsets based on a curriculum for the training scheduler. Finally, we compared the performance of our CURriculum Learning-based AutoML system, which we dub CurL-AutoML, against the original Auto-sklearn system, on twelve datasets built for classification tasks, by means of the AutoML Benchmark framework presented in [12].

In general, the results show that our approach shows improvements over the baseline, Auto-sklearn, on most of the selected datasets.

The main contributions of our work are:

- The CurL-AutoML, a new approach for incorporating Curriculum Learning in an AutoML tool.
- A comparison of CL and AntiCL strategies in the context of Machine Learning Automation.
- A comparison of a distance-based (kDN) and a probabilistic (GMM) difficulty measurers for curriculum design.

II. BACKGROUND

AutoML is the process of automating any step of the ML pipeline. It emerged from the will to reduce the necessary expertise to apply machine learning, making it more accessible for non-experts, such as domain scientists, that do not have a deep knowledge about the technologies behind it [2].

Currently, AutoML techniques are usually found in three steps of the traditional ML pipeline: data preparation, feature engineering, and algorithm selection/configuration.

Among the steps of a machine learning pipeline, algorithm selection and configuration are arguably the most investigated in AutoML. It is easy to understand its motivations, considering that different ML algorithms have numerous potential configurations, each one with its hyperparameters to be tuned. The choice of which one to use becomes a challenging problem, mainly for non-expert users.

The problem of simultaneously selecting an algorithm and tuning its hyperparameters to optimize one or more objectives was first introduced by [13] and defined as the combined algorithm selection and hyperparameter optimization (CASH) problem. Solving CASH for AutoML is time-consuming. One way to cope with this problem is through multi-fidelity optimization methods [14]. These methods, such as Successive Halving, speed up optimization employing low-fidelity approximations of the actual loss function, which must be minimized. Successive Halving uses adaptive resource allocation and early-stopping, terminating the evaluation of probably bad-performing configurations early. Although useful, early-stopping may produce sub-optimal results. It means that a configuration that would yield the best final results may be discarded in favor of others that had a better initial performance on a low-fidelity approximation but ended up yielding worse final results.

In this work, we applied curriculum strategies to Successive Halving to alleviate such problems by guiding the search towards better regions in the search space, aiming to find better-performing ML pipelines with efficiency. Although our proposed CL approach can theoretically be applied to any AutoML tool, we chose Auto-sklearn to perform our experiments.

Thus, the next subsections review the necessary information regarding the fundamental components that design our proposal, CurL-AutoML: Auto-sklearn, Successive Halving, and Curriculum Learning, respectively.

A. Auto-sklearn

Auto-sklearn [11] is an AutoML tool developed for building classification and regression pipelines.

By default, Auto-sklearn uses meta-learning to start the optimization algorithm in a region of the search space which is potentially close to the optimum (warm-start).

Auto-sklearn pipeline begins with a hardcoded data preparation step, which includes methods such as imputation and categorical encoding. Subsequently, there is a feature engineering step. Next, an ML algorithm is selected and has its hyperparameters optimized by the Sequential Model-based Algorithm Configuration (SMAC) [15] algorithm.

SMAC is a Sequential Model-Based Optimization (SMBO) [15] framework, that is, a stochastic optimization framework that can explicitly work with both categorical and continuous hyperparameters. It can also exploit conditional spaces, such as when a hyperparameter may only be relevant if one or more hyperparameters assumes a specific value [16]. A common approach to apply SMBO is by Bayesian optimization [17], which iteratively fits a surrogate model to observations of the target function. Then, it uses an acquisition function to determine the following candidate points based on the predictive distribution of the surrogate model. With the increase in the number of observations, the surrogate model's predictive distribution improves. Consequently, the algorithm has a better assumption of which regions in the parameter space are worth exploring and which are not. SMAC is a popular tree-based approach for the surrogate model and is the CASH solver for Auto-sklearn.

Furthermore, Auto-sklearn applies ensemble learning to make the final result more robust against overfitting. During the optimization process, the tool stores the models evaluated in the search space. Then, it applies ensemble selection [18], a model post-processing method that starts from an empty ensemble and, following a greedy approach, iteratively adds the model that maximizes ensemble validation performance, constructing an ensemble of the previously trained models.

B. Successive Halving

Successive Halving (SH) [19] is a bandit-based strategy, applied for multi-fidelity algorithm selection that tries to find the best algorithm given a finite set of algorithms and taking into account the low-fidelity approximations of their performance.

As a result, SH queries all available algorithms for a certain initial budget (which might be a fraction of the training data, a number of iterations, etc.), discarding the worst-performing half, doubling the budget, and iterating this process until only one algorithm remains.

More specifically, it removes the worst $\frac{n-1}{n}$ algorithms to then multiply the budget for the remaining algorithms by n , where n is a hyperparameter (the default value for SH is 2) [2].

A configuration algorithm, such as SMAC, has a fundamental mechanism called intensification. This intensification mechanism controls how many evaluations to perform on each configuration and decides when a configuration becomes the new current best-known configuration. [15].

One of the capabilities of Auto-sklearn is to use SH as the intensification mechanism for the SMAC optimizer. It combines Bayesian optimization with SH (BOSH), resulting in a robust and efficient hyperparameter optimization algorithm.

C. Curriculum Learning

Curriculum Learning consists of training models from the easiest to the hardest data. Its benefits may include faster model training, convergence, and better generalization capacity. In contrast, anti-curriculum learning (Anti-CL) consists of training an ML model from the hardest to the easiest data.

For applying a CL strategy, we need to define two components [6]: (i) the difficulty measurer, to generate our curriculum by estimating the difficulty of each training example; and (ii) the training scheduler, to control the pace and sequence of data presented to the model during training.

A predefined CL has both the difficulty measurer and the training scheduler designed by prior human knowledge with no data-driven models or algorithms involved. In contrast, an automatic CL has at least one of the components involving an algorithm.

There are different approaches to define a difficulty measurer for a predefined CL, and it is usually tailored to the task. A predefined training scheduler can be discrete or continuous, and is usually task-agnostic [6]. A discrete training scheduler adjusts the training data after every fixed number of epochs or convergence on the current data subset. In contrast, continuous schedulers adjust the training data subset at every epoch.

Wang et al. [6] summarize four of the major strategies for automatic CL:

- **Transfer Teacher:** A pre-trained model acts as the teacher and measures the difficulty of training examples according to its performance on them, which is later used for training a student algorithm.
- **Self-Paced Learning (SPL):** The student algorithm also acts as a teacher, measuring the difficulty of training examples according to its losses on them.
- **Reinforcement Learning Teacher:** A reinforcement learning (RL) model act as the teacher to perform dynamic data selection according to the feedback from the student.
- **Other Automatic CL:** Methods that include various automatic CL strategies except the above.

III. CURL-AUTOML

In this section, we describe our approach to apply CL strategies in an AutoML system. Since a predefined CL requires human intervention and it is opposite to the primary goal of AutoML, we chose to follow the automatic CL strategy. More specifically, we adopt a Transfer Teacher strategy.

As automatic difficulty measurers, we utilized methods based on instance hardness (IH). Hardness ordering is the application of IH methods to order examples by difficulty for classification, and was first introduced by Smith and Martinez [20].

In this paper, we use the k-Disagreeing Neighbors approach to estimate IH and generate a curriculum by hardness ordering, and we present a probabilistic-based method for doing so:

- **k-Disagreeing Neighbors (kDN):** kDN measures the local overlap between an instance and its nearest neighbors. In this method, an instance is considered hard to classify when there is an overlap of different classes in its region of competence. The kDN of an instance is the percentage of the k nearest neighbors (using Euclidean distance) of that instance with a different target class value [10].
- **Gaussian Mixture Model (GMM):** In this work, we proposed and investigated the use of GMM [21] as an estimator of IH. Our approach consists of fitting a GMM to a dataset, then, for every instance, we extract the probabilities of belonging to one the N Gaussian distributions, where N is the number of target classes. Thus, the IH will be defined according to the probability of an instance not being generated by a Gaussian distribution. Considering that the number of Gaussians is defined according to the number of target classes, the instances that are in the decision boundaries between Gaussians will be deemed to have a greater degree of difficulty than those closer to a Gaussian's centroid.

After the hardness ordering, it is necessary a training scheduler to apply CL. In Auto-sklearn, when using the BOSH optimizer, subsets of training data can be taken as a budget. By default, these subsets are randomly sampled from the training data. We adapted this sampling strategy to select subsamples of data according to a curriculum strategy, such as CL or AntiCL. Therefore, in our approach, CurL-AutoML, the SH procedure used in the BOSH algorithm acts as a predefined training scheduler for Auto-sklearn.

For the CL approach, the relatively easiest examples are sampled first, then SH evaluates algorithms configurations on it, discarding the worst-performing half. In the next iterations, SH will progressively sample harder data and repeat the process, until the budget reaches its limit and one configuration is left. In the AntiCL approach, in turn, the process starts from the hardest examples and goes on decreasing the difficult of the instances until the end. In contrast to the general transfer teacher curriculum design, our training scheduler does not samples at each training epoch, but at each SH iteration. Moreover, it does not only involves the training of a single algorithm, but multiple algorithms configurations.

Figure 1 illustrates the design of our proposal. First the selected dataset is utilized to pretrain a difficulty measurer model, which then is used to assign a difficulty score to each instance in the dataset. Next, we select one curriculum learning strategy, which could be the traditional curriculum (easy instances to hard instances) or the anti-curriculum (hard instances to easy instances). Then, the AutoML optimization process starts. During optimization, the Successive Halving procedure used in BOSH acts as the training scheduler and uses the previously sorted data to train and evaluate multiple algorithms configurations. Finally, the best ML pipeline

constructed during optimization is outputted.

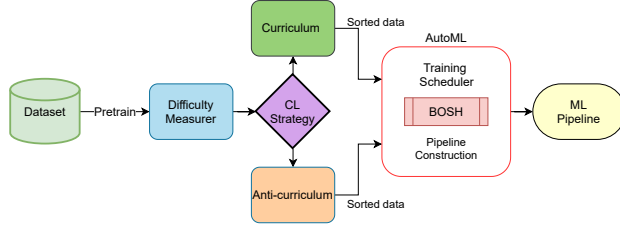


Fig. 1. Curriculum learning-based AutoML design.

IV. RELATED WORK

Several AutoML systems were developed in the last decade [13], [22]–[26]. However, solving the AutoML problem is still time-consuming. Therefore, different approaches need to be proposed to tackle this complex search space and find good performing ML pipelines efficiently. Some previous approaches aim to speed up AutoML through methods such as meta-learning and multi-fidelity optimization [3], [27]. Auto-sklearn achieved a boost in the search for the best ML pipeline by warm-starting the Bayesian optimization procedure via meta-learning [27].

Curriculum Learning was first proposed by [5], where the authors conducted an empirical investigation on the use and effects of CL for supervised visual and language tasks. The difficulty of examples was determined by the loss value of a pre-trained model.

Since its proposal, CL has been investigated on a plenty of ML applications, such as reinforcement learning [28], [29], natural language processing [30], [31], healthcare prediction [32], computer vision [33].

However, to the best of our knowledge, Guo et al. [9] is the only previous work that investigates the use of CL on AutoML, more specifically Neural Architecture Search (NAS). Guo et al. [9] proposed a curriculum search method that starts from a small search space and gradually incorporates the learned knowledge to guide the search in a large space. Extensive experiments on CIFAR-10 and ImageNet demonstrated significant improvements in the search efficiency and architecture performance over previous NAS methods.

In contrast to the previously described work, in this paper we focus on applying CL strategies on AutoML for ML pipelines that do not include deep neural networks. The previous empirical investigations on CL have shown improvements in multiple ML domains. In our work, we investigated the use of CL strategies on AutoML tasks, while still leveraging the benefits of multi-fidelity optimization and meta-learning, by adapting the SH procedure utilized by the CASH solvers implemented in Auto-sklearn.

V. EXPERIMENTAL EVALUATION

In this section, we describe our experiments with the proposed CurL-AutoML. All tests were performed on a computer with 12 cores Intel Xeon E3 - 12xx v2 (Ivy Bridge), which

operated at 2.4 GHz, and operational memory of 64 GB. Code for all experiments is in the GitHub repository at <https://github.com/lucasnil/curl-automl>.

A. Experimental Setup

In total, we selected twelve datasets for experimentation from the Open ML repository. Table I presents the datasets. MiniBooNE, Jasmine, Amazon, Australian, Blood transfusion, and Christine, are datasets that represent binary classification problems, while Covtype, Helena, Fashion-MNIST, CNAE-9, Connect-4, and Dilbert, represent multi-class classification problems.

TABLE I
SELECTED OPENML DATASETS.

Dataset	Instances	Features	Classes	OpenML ID
Covtype	581,012	55	7	1596
Helena	65,196	28	100	41169
MiniBooNE	130,064	51	2	41150
Jasmine	2,984	145	2	41143
Amazon	32,769	10	2	4135
Australian	690	15	2	40981
Fashion-MNIST	70,000	785	10	40996
Blood transfusion	748	5	2	1464
Christine	5418	1637	2	41142
CNAE-9	1080	857	9	1468
Connect-4	67,557	43	3	40668
Dilbert	10,000	2001	5	41163

First, we generated two different curriculums for each dataset, one by kDN and another via GMM.

The execution of the experiments was carried out through the AutoML benchmark framework introduced by [12]. We used this framework to run and evaluate the performance of our proposed CurL-AutoML, using the BOSH optimizer (with CL and AntiCL strategies) on the selected datasets. We performed the same experiments for Auto-sklearn and compared the results with those achieved by CurL-AutoML. The benchmarking framework supports a broad range of measures. In this work, we chose log loss for both binary classification and multi-class classification problems. The measures are estimated with five-fold cross-validation.

Since CurL-AutoML is an adaption of Auto-sklearn, both AutoML tools were used with the same search spaces. We fixed the number of 12 processing units, 64GB memory, and a run time of 1 hour per cross-validation fold as hyperparameters that specified available resources.

B. Results

In this section, we describe the results we obtained in our experimentation. All curriculum approaches were executed CurL-AutoML using the BOSH optimizer, while the standard Auto-sklearn carried out the “No curriculum” approach.

The total time for generating a curriculum via kDN and GMM for each dataset is presented in Table II. In general, both difficulty measurers could generate a curriculum in a feasible time, but kDN had the lowest average time.

TABLE II
TIME IN SECONDS FOR GENERATING A CURRICULUM VIA KDN OR GMM,
FOR EACH DATASET.

Dataset	Difficulty measurer	Time in seconds
Coverttype	kDN	400
	GMM	129
Helena	kDN	4
	GMM	504
MiniBooNE	kDN	20
	GMM	110
Jasmine	kDN	0.20
	GMM	0.26
Amazon	kDN	1.2
	GMM	0.56
Australian	kDN	0.04
	GMM	0.04
Fashion-MNIST	kDN	49
	GMM	1308
Blood transfusion	kDN	0.01
	GMM	0.07
Christine	kDN	1.7
	GMM	8.9
CNAE-9	kDN	0.06
	GMM	4
Connect-4	kDN	5.9
	GMM	4.9
Dilbert	kDN	3
	GMM	60

Table III presents the average log loss scores and standard deviation on every selected dataset for each curriculum approach and optimizer. Curriculum approaches achieved minor improvements over the No curriculum baseline on ten of the twelve selected datasets, the AntiCL and CL approaches with a curriculum generated via kDN achieved improvements in seven datasets, while those generated via GMM were better in the remaining three. The AntiCL-kDN is the approach with the most number of improvements, followed by CL-kDN. The baseline was not surpassed on the Blood transfusion (binary classification) dataset, and the CNAE-9 (multiclass classification) dataset.

VI. FINAL REMARKS

We proposed and compared the use of different CL strategies on an AutoML system to accelerate the exploration of the search space and find good performing machine-learned models. Our proposal, CurL-AutoML, has the design of an automatic CL framework. More specifically, we adopted a Teacher Transfer strategy, using instance hardness methods as our automatic difficulty measurers, and adapted the SH procedure implemented in the AutoML system Auto-sklearn to act as our predefined training scheduler.

The experiments show that using a curriculum strategy can guide AutoML to better results in most evaluated scenarios. The most successful approaches utilized the anti-curriculum strategy. We assume that by evaluating configuration first on the most difficult data, the SH procedure can select more robust algorithm configurations, which otherwise would be discarded.

In some cases, the kDN-based approaches achieved worse results, indicating that kDN may not be adequate to be used

as a difficult measurer in specific scenarios. For this reason, we intend to study the relations between difficult measurers and the data space to understand this phenomenon better and propose techniques to avoid these pitfalls.

Since the curriculum generation time for each dataset was not much costly, it is feasible to add a pre-training step in the AutoML pipeline to benefit from a CL strategy. Regarding the difficulty measurers, kDN was the fastest, and its curriculums mostly outperformed those generated by GMM.

Even after our experimental evaluation, there still a need to investigate the effects of CL strategies for AutoML in a broader range of datasets and ML tasks. To implement a Transfer Teacher method, we employed kDN and GMM as teacher algorithms, each of which has its own set of hyperparameters that can be tweaked. Thus, future work may investigate the effects of the proposed difficulty measurers hyperparameters on the final generated curriculum.

Besides, we intend to investigate the mechanisms that make one approach achieve better results than others in each case. Depending on the results of this exploration, we may propose a technique to automatically select or ensemble different strategies.

Moreover, there is still room for improvement regarding the proposed training schedulers, concerning its pace (halving factor) and budget size.

REFERENCES

- [1] C. Yang, J. Fan, Z. Wu, and M. Udell, "Automl pipeline selection: Efficiently navigating the combinatorial space," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1446–1456. [Online]. Available: <https://doi.org/10.1145/3394486.3403197>
- [2] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [3] L. Parmentier, O. Nicol, L. Jourdan, and M. Kessaci, "Tpot-sh: A faster optimization algorithm to solve the automl problem on large datasets," in *2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI)*, 2019, pp. 471–478.
- [4] J. L. Elman, "Learning and development in neural networks: the importance of starting small," *Cognition*, vol. 48, no. 1, pp. 71–99, 1993. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0010027793900584>
- [5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, ser. ICML '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 41–48. [Online]. Available: <https://doi.org/10.1145/1553374.1553380>
- [6] X. Wang, Y. Chen, and W. Zhu, "A comprehensive survey on curriculum learning," *arXiv preprint arXiv:2010.13166*, 2020.
- [7] T. Kocmi and O. Bojar, "Curriculum learning and minibatch bucketing in neural machine translation," in *Proceedings of the International Conference Recent Advances in Natural Language Processing, RANLP 2017*, 2017, pp. 379–386.
- [8] X. Zhang, P. Shapiro, G. Kumar, P. McNamee, M. Carpuat, and K. Duh, "Curriculum learning for domain adaptation in neural machine translation," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 1903–1915.
- [9] Y. Guo, Y. Chen, Y. Zheng, P. Zhao, J. Chen, J. Huang, and M. Tan, "Breaking the curse of space explosion: Towards efficient nas with curriculum search," in *International Conference on Machine Learning*. PMLR, 2020, pp. 3822–3831.

TABLE III
AVERAGE LOG LOSS SCORES AND STANDARD DEVIATION ON ALL SELECTED DATASETS FOR EVERY CURRICULUM STRATEGY USING BOSH.

Dataset	No CL	AntiCL-kDN	AntiCL-GMM	CL-kDN	CL-GMM
Coverttype	0.2572 (0.04)	0.2133 (0.03)	0.2502 (0.04)	0.2413 (0.04)	0.2456 (0.04)
Helena	2.5589 (0.01)	2.5563 (0.01)	2.5605 (0.01)	2.5532 (0.01)	2.5585 (0.02)
MiniBooNE	0.2627 (0.19)	0.1393 (0.006)	0.2818 (0.16)	0.1337 (0.005)	0.1340 (0.004)
Jasmine	0.4003 (0.04)	0.3905 (0.04)	0.3965 (0.02)	0.3942 (0.04)	0.3918 (0.02)
Amazon	0.1595 (0.06)	0.1594 (0.09)	0.1857 (0.04)	0.1611 (0.01)	0.1624 (0.007)
Australian	0.4446 (0.18)	0.4308 (0.07)	0.4068 (0.05)	0.3930 (0.02)	0.4163 (0.1)
Fashion-MNIST	0.2711 (0.006)	0.2696 (0.005)	0.2704 (0.004)	0.2721 (0.007)	0.2698 (0.004)
Blood transfusion	0.5520 (0.09)	0.5900 (0.1)	0.5562 (0.04)	0.5700 (0.06)	0.6006 (0.08)
Christine	0.5364 (0.04)	0.5336 (0.01)	0.5269 (0.02)	0.5860 (0.03)	0.5537 (0.06)
CNAE-9	0.1726 (0.05)	0.1856 (0.07)	0.1866 (0.06)	0.2095 (0.05)	0.1930 (0.09)
Connect-4	0.3681 (0.08)	0.3622 (0.01)	0.3524 (0.01)	0.3574 (0.01)	0.3604 (0.01)
Dilbert	0.0375 (0.01)	0.0364 (0.01)	0.0373 (0.01)	0.0357 (0.01)	0.0363 (0.01)

- [10] M. R. Smith, T. Martinez, and C. Giraud-Carrier, "An instance level analysis of data complexity," *Mach. Learn.*, vol. 95, no. 2, p. 225–256, May 2014.
- [11] M. Feurer and F. Hutter, "Towards further automation in autml," in *ICML AutoML workshop*, 2018.
- [12] P. Gijsbers, E. LeDell, S. Poirier, J. Thomas, B. Bischl, and J. Vanschoren, "An open source autml benchmark," in *6th ICML Workshop on Automated Machine Learning*, 2019.
- [13] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-weka: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 847–855. [Online]. Available: <https://doi.org/10.1145/2487575.2487629>
- [14] M. Giselle Fernández-Godino, C. Park, N. H. Kim, and R. T. Haftka, "Issues in deciding whether to use multifidelity surrogates," *AIAA Journal*, vol. 57, no. 5, p. 2039–2054, May 2019. [Online]. Available: <http://dx.doi.org/10.2514/1.J057750>
- [15] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proceedings of the 5th International Conference on Learning and Intelligent Optimization*, ser. LION'05. Berlin, Heidelberg: Springer-Verlag, 2011, p. 507–523.
- [16] L. Kotthoff, C. Thornton, H. H. Hoos, F. Hutter, and K. Leyton-Brown, "Auto-weka: Automatic model selection and hyperparameter optimization in weka," in *Automated Machine Learning: Methods, Systems, Challenges*, F. Hutter, L. Kotthoff, and J. Vanschoren, Eds. Cham, Switzerland: Springer International Publishing, 2019, pp. 81–95.
- [17] E. Brochu, V. M. Cora, and N. de Freitas, "A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning," 2010.
- [18] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes, "Ensemble selection from libraries of models," in *Proceedings of the Twenty-First International Conference on Machine Learning*, ser. ICML '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 18. [Online]. Available: <https://doi.org/10.1145/1015330.1015432>
- [19] K. Jamieson and A. Talwalkar, "Non-stochastic best arm identification and hyperparameter optimization," in *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, A. Gretton and C. C. Robert, Eds., vol. 51. Cadiz, Spain: PMLR, 09–11 May 2016, pp. 240–248. [Online]. Available: <http://proceedings.mlr.press/v51/jamieson16.html>
- [20] M. R. Smith and T. Martinez, "A comparative evaluation of curriculum learning with filtering and boosting in supervised classification problems," *Computational Intelligence*, vol. 32, no. 2, pp. 167–195, 2016.
- [21] D. Reynolds, *Gaussian Mixture Models*. Boston, MA: Springer US, 2009, ch. 1, pp. 659–663.
- [22] B. Komer, J. Bergstra, and C. Eliasmith, "Hyperopt-sklearn: automatic hyperparameter configuration for scikit-learn," in *ICML workshop on AutoML*, vol. 9, 2014, p. 50.
- [23] R. S. Olson, N. Bartley, R. J. Urbanowicz, and J. H. Moore, "Evaluation of a tree-based pipeline optimization tool for automating data science," in *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, ser. GECCO '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 485–492. [Online]. Available: <https://doi.org/10.1145/2908812.2908918>
- [24] T. Swearingen, W. Drevo, B. Cyphers, A. Cuesta-Infante, A. Ross, and K. Veeramachaneni, "Atm: A distributed, collaborative, scalable system for automated machine learning," in *2017 IEEE International Conference on Big Data (Big Data)*, 2017, pp. 151–162.
- [25] A. G. C. de Sá, W. J. G. S. Pinto, L. O. V. B. Oliveira, and G. L. Pappa, "Recipe: A grammar-based framework for automatically evolving classification pipelines," in *Genetic Programming*, J. McDermott, M. Castelli, L. Sekanina, E. Haasdijk, and P. García-Sánchez, Eds. Cham: Springer International Publishing, 2017, pp. 246–261.
- [26] B. Chen, H. Wu, W. Mo, I. Chattopadhyay, and H. Lipson, "Autostacker: A compositional evolutionary learning system," in *Proceedings of the Genetic and Evolutionary Computation Conference*, ser. GECCO '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 402–409. [Online]. Available: <https://doi.org/10.1145/3205455.3205586>
- [27] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, ser. NIPS'15. Cambridge, MA, USA: MIT Press, 2015, p. 2755–2763.
- [28] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel, "Reverse curriculum generation for reinforcement learning," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., vol. 78. PMLR, 13–15 Nov 2017, pp. 482–495. [Online]. Available: <http://proceedings.mlr.press/v78/florensa17a.html>
- [29] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum learning for reinforcement learning domains: A framework and survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.
- [30] E. A. Platanios, O. Stretcu, G. Neubig, B. Poczos, and T. Mitchell, "Competence-based curriculum learning for neural machine translation," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 1162–1172.
- [31] C. Xu, B. Hu, Y. Jiang, K. Feng, Z. Wang, S. Huang, Q. Ju, T. Xiao, and J. Zhu, "Dynamic curriculum learning for low-resource neural machine translation," in *Proceedings of the 28th International Conference on Computational Linguistics*. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020, pp. 3977–3989. [Online]. Available: <https://www.aclweb.org/anthology/2020.coling-main.352>
- [32] R. El-Bouri, D. Eyre, P. Watkinson, T. Zhu, and D. Clifton, "Student-teacher curriculum learning via reinforcement learning: Predicting hospital inpatient admission location," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 2848–2857.
- [33] Y. Wang, W. Gan, J. Yang, W. Wu, and J. Yan, "Dynamic curriculum learning for imbalanced data classification," in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 5016–5025.